Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	16/02/2026
Team ID	69a7fb1de8fb1cec7726dd56
Project Name	Contact Manager
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
1	User Authentication and Authorization	• User registration with name, email, and password • User login with email and password • JWT token generation and management • Password encryption using bcrypt • Session management and auto-logout on token expiration • Protected routes requiring authentication • User logout functionality
2	Contact Management (CRUD Operations)	• Create new contact with name, email, phone, notes, and tags • View all contacts for logged-in user • View individual contact details • Update existing contact information • Delete contact with confirmation prompt • Validate contact data (email format, required fields) • Display contact count and metadata

3	Search and Discovery	• Real-time search across contact names • Search by email addresses • Search by tags • Case-insensitive search functionality • Display search results count
4	Data Validation and Error Handling	• Server-side validation for all API endpoints • Email format validation • Required field validation • Display validation errors to user • API error handling and user-friendly messages • Prevent duplicate email registration
5	Contact Information Management	• Store contact name (required) • Store contact email (optional, validated) • Store contact phone number (optional) • Store contact notes/description (optional, max 1000 chars) • Store contact tags (optional, max 10 tags) • Timestamp for contact creation • Timestamp for last update
6	User Profile and Account	• Get current user information endpoint • User-specific data isolation • Each user accesses only their own contacts • User ID association with all contacts
7	Data Persistence and Storage	• Store user data in MongoDB database • Store contacts in MongoDB database • Maintain referential integrity (userId foreign key)

		• Automatic timestamp management • Database indexing for performance • Secure password storage (hashed) • LocalStorage for JWT token on client

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
1	Security	All API requests must be secured using JWT authentication. The app should prevent unauthorized access to user data. Passwords must be hashed using bcrypt with salt rounds of 10 or higher. SQL/NoSQL injection attacks should be prevented through input validation and parameterized queries. CORS properly configured to allow only authorized origins. Sensitive data (passwords) never exposed in API responses. HTTPS enforced in production environments.
2	Reliability	The system should handle errors gracefully without crashing. Database connections should be resilient with automatic reconnection. Failed API requests should be handled with appropriate error messages.
3	Performance	API response times should

		average under 200ms. Database queries optimized with appropriate indexing. Client-side rendering should be smooth (60fps animations). Bundle size kept under 500KB for fast loading.
4	Scalability	The app should handle increasing numbers of users and contacts efficiently without performance degradation. The architecture should support future feature expansion. Should support at least 10,000 concurrent users. Individual users should be able to store up to 10,000 contacts without performance issues. Database should scale horizontally
5	Maintainability	The codebase should be well-organized, documented, and easy to maintain. Code should follow consistent style guidelines and best practices. Functions should be modular and single-purpose. Comments should explain complex logic. API endpoints should be clearly documented. Database schema should be well-defined. Version control with meaningful commit messages. Separate development, staging, and production environments.
6	Compatibility	The application should work across different browsers and devices. Support for Chrome (latest 2 versions), Firefox (latest 2 versions), Safari (latest 2 versions), Edge (latest 2 versions). Mobile support for iOS 12+ and Android 8+. Responsive design working on screen

		sizes from 320px to 2560px. No dependency on specific operating systems. API compatible with different client types (web, mobile, future integrations).
7	Data Privacy and Compliance	User data should be handled in accordance with privacy regulations. Each user can only access their own data. No sharing of user data with third parties. Data deletion on account removal (future feature). Clear privacy policy available. Compliance with GDPR principles (data minimization, purpose limitation). User consent for data processing.